

Java Programming 3

Thymeleaf



Agenda this week

Last week - project Review

Thymeleaf

Agenda this week



Last week - project Review

Thymeleaf

Last week - Project Review

- Last week review:
 - Application Context - Logging - Intro SpringMVC
- Project Remarks:
 - Process the inputfields of the form?
 - You could create a special container class for that: "ViewModel"

```
public class AuthorViewModel {  
    private String name;  
    private String birthday;  
    private String nationality;  
    ...  
}
```

```
@PostMapping("/add")  
public String processAddAuthorForm(AuthorViewModel authorViewModel) {  
    log.debug("Processing addauthor form...");  
    LocalDate birthday = LocalDate.parse(authorViewModel.getBirthday());  
    authService.addAuthor(authorViewModel.getName(), birthday,  
        authorViewModel.getNationality());  
}
```

More tips and tricks on how to process the form in your Controller later, when we look at Controllers in detail!



Agenda this week

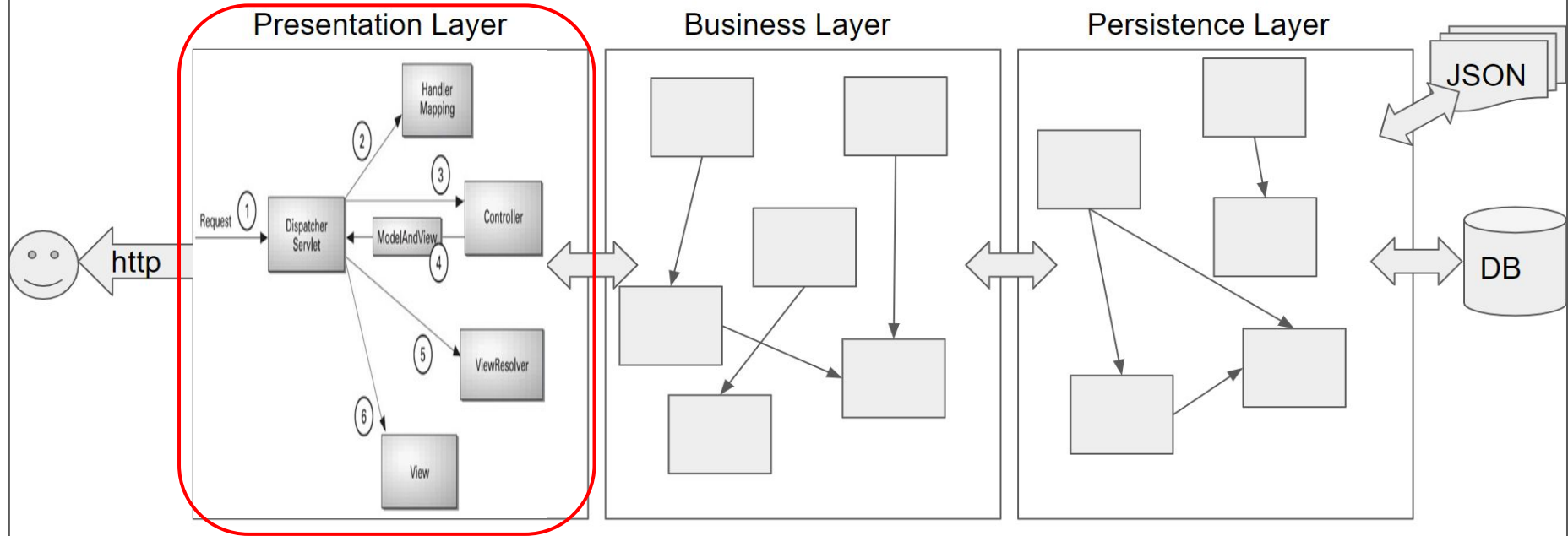


Last week - project Review

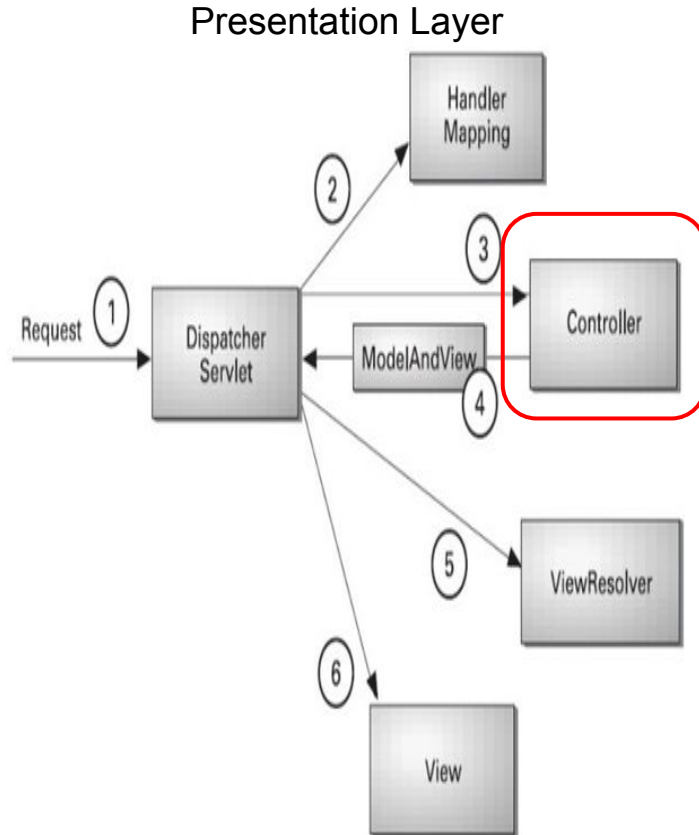
Thymeleaf

Recap: the Presentation Layer

A 3-tier web application using the Java Spring Framework



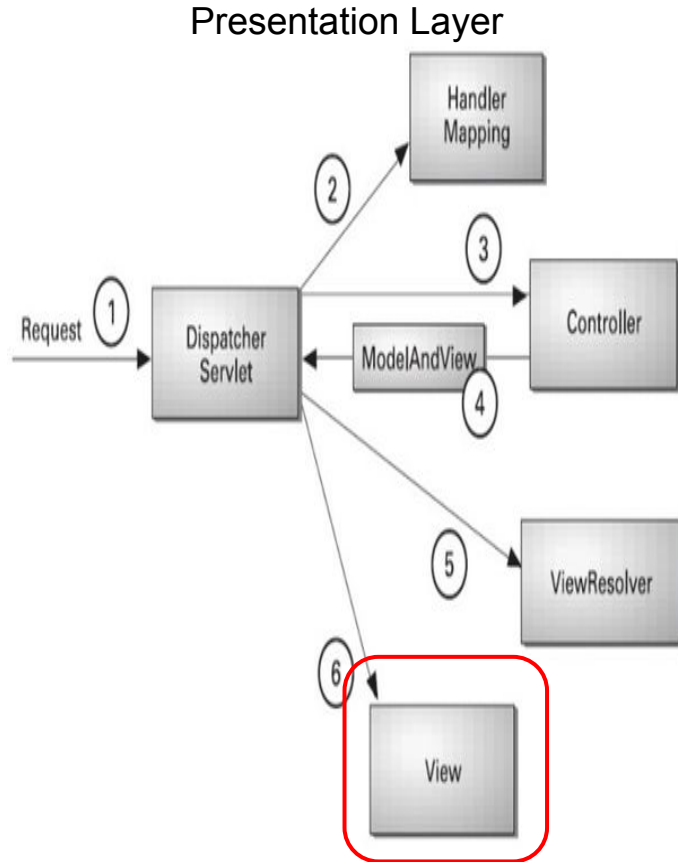
Presentation Layer: Controllers and Views



Controllers:

- We will write Controllers using Java (with `@Controller` annotation).
- We typically create a controller for each 'entity' in the domain, although it depends on the application...
- Each controller can then have methods for getting views with (list of) entity/ies and for processing forms for adding entities and/or updating and/or deleting.
- Controller talks to the service layer
- The controller passes data and the logical name of the view to the DispatcherServlet, who then runs the (Thymeleaf) view with this data

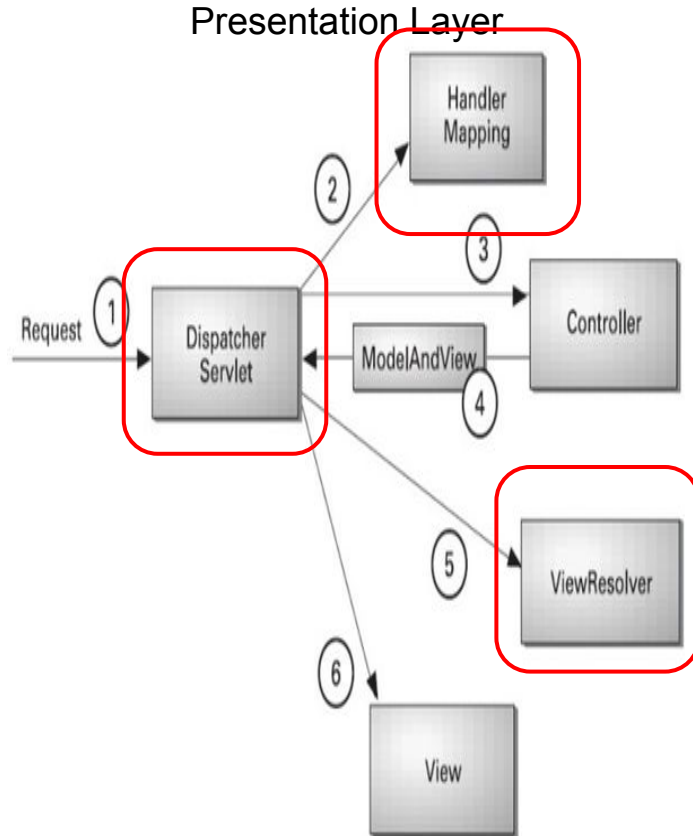
Presentation Layer: Controllers and Views



Views:

- We will write Views using Thymeleaf templates
- We create a Thymeleaf view for each page of our application
- The Controller decides what view is necessary and passes on data to the view (via the DispatcherServlet)
- Thymeleaf template generates the HTML using the data it received from the Controller

Presentation Layer: Controllers and Views



DispatcherServlet, HandlerMapping, ViewResolver

- These objects are automatically created by the Spring framework and added to the Spring container
- They are configured for Thymeleaf because Spring Boot finds the Thymeleaf dependency in the classpath. (This process is called “autoconfiguration”)

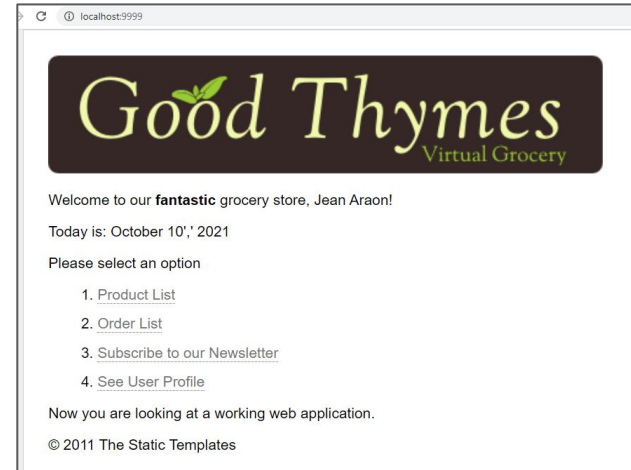
Thymeleaf



- a server-side Java template engine
- main goal is to bring *natural templates* — HTML that can be correctly displayed in browsers and also work as static prototypes
- With modules for Spring Framework

Thymeleaf Spring example project

- Clone the [Good Thymes Virtual Grocery example project](#)
 - This project is used as an example in the thymeleaf documentation:
 - [Using Thymeleaf](#)
 - Thymeleaf and Spring:
<https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html>
 - Run it!

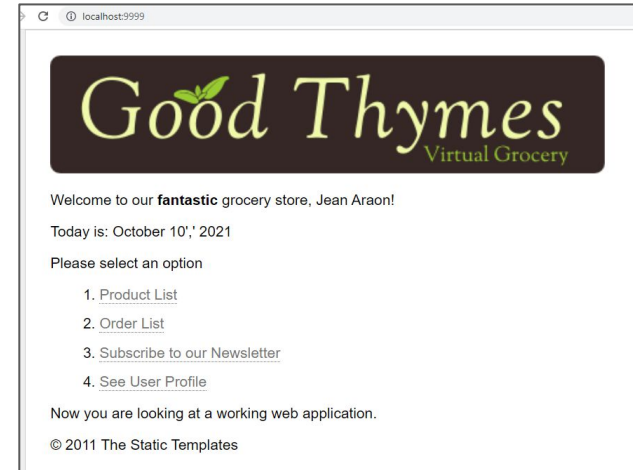
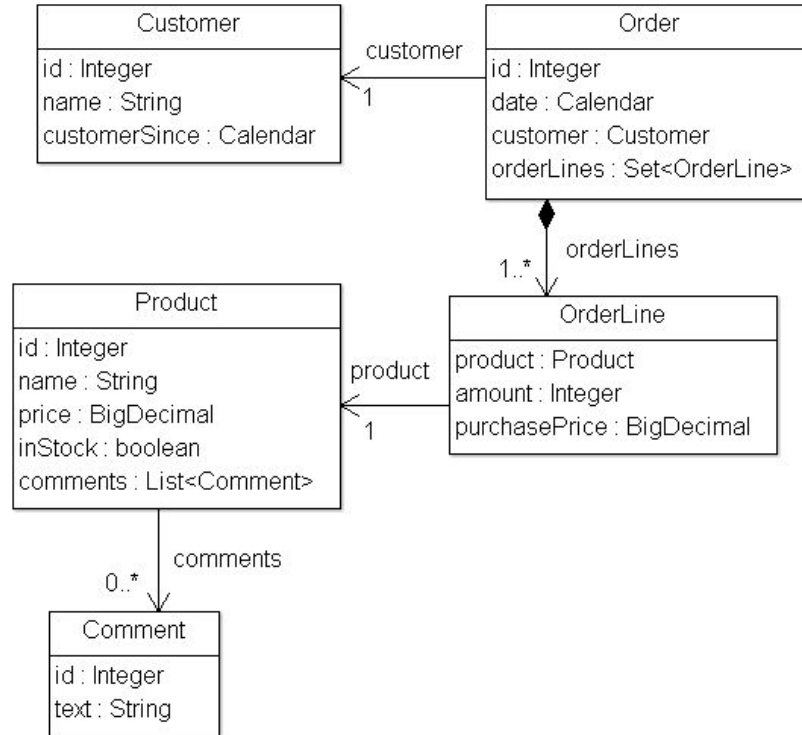




Exercise 1.1

- Run the Good Thymes application - Test it!
- On what port does the web application run?
- Inspect the Java code
 - Draw a UML diagram for the Domain Model
 - Inspect the different layers of the application
 - How do they differ from our implementations?
 - What would you change?

Virtual Grocery domain model





Exercise 1.2: inspect home.html

- Look up and explain each of the following th:attributes in detail!
 - In <link>:
 - th:href="@{/css/gtvg.css}"
 - In :
 - th:src="@{/images/...}"
 - th:alt-title="#{logo}"
 - In <p>:
 - th:utext="#{home.welcome(\${session.user.name})}"
 - In :
 - th:with="df=#{date.format}"
 - th:text="\${#temporals.format(session.today,df)}"
 - In <a>:
 - th:href="@{/product/list}"
 - In <div>:
 - th:insert=~{footer::copy}"



Exercise 1.3: inspect product/list.html

- Look up and explain each of the following th:attributes in detail!
 - In <tbody>:
 - `th:remove="all-but-first"`
 - In <tr>:
 - `th:each="prod : ${prods}"`
 - `th:class="${prodStat.odd}? 'odd'"`
 - In <td>:
 - `th:text="${prod.inStock}? #{true} : #{false}"`
 - In :
 - `th:text="${#lists.size(prod.comments)}"`
 - In <a>:
 - `th:unless="${#lists.isEmpty(prod.comments)}"`

Five different *Expression* types

- Tutorial: [Getting started in 5 minutes](#)
 - $@\{\dots\}$ → URL expressions
 - $\#\{\dots\}$ → Message expressions
 - $\$\{\dots\}$ → Variable expressions
 - $\sim\{\dots\}$ → Fragment expressions
 - $*\{\dots\}$ → Selection variable expressions



@{...}: URL Expressions

- Will be replaced by path to the correct resource
- Can refer to location in static or in templates folder (or other website)
- Can have parameters: will be added to the URL as URL parameters
- Examples:

```
<link rel="stylesheet" type="text/css" media="all"  
  th:href="@{/css/gtvgl.css}" />
```

```

```

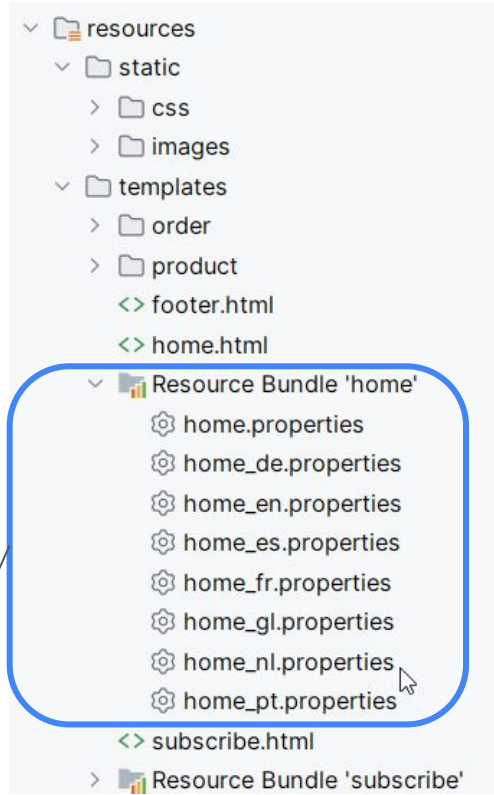
```
<a th:href="@{/order/details(id=${orderId}, type=${orderType})}">
```

#{...} → Message expressions

- They refer to a value in a resource bundle
 - Resource Bundle = set of .properties files, one for each language, containing strings
 - IntelliJ has a plugin that simplifies working with resource bundles
 - Default location: resources/messages.properties (messages_nl.properties, ...)
 - Or add location to application.properties, e.g.:

```
spring.messages.basename=templates/home
```

```
home.welcome=Welkom bij onze <b>kruidenier</b>, {0}!  
logo=Logo Good Thymes Virtual Grocery  
date.format=dd MMMM', ' yyyy
```



`#{...}` → Message expressions

- Message expressions can have parameters
- Examples:

```

```

```
<p th:utext="#{home.welcome(${session.user.name})}">Welcome!</p>
```

Unescaped text:
can contain HTML that will
be added to the page

`${...}` → variable expressions

- In variable expressions you can use the attributes that were added to the Model object in the Controller class.
- You can also access Thymeleaf predefined [Expression Objects](#) using the # inside `${...}`
- Examples:

```
@Controller
public class ProductListController {

    @RequestMapping("/product/list")
    public String product(Model model){
        model.addAttribute("prods",prods.findAll());
        return "product/list";
    }
}
```

```
<tr th:each="prod : ${prods}" th:class="${prodStat.odd}? 'odd'">
```

```
<span th:text="${#lists.size(prod.comments)}">2</span>
```

~{...} → Fragment expressions

- You can refer to reusable HTML snippets called “fragments”
- The fragments reside in another file and have the `th:fragment` attribute
- Typically used for headers and footers.
- They can also have parameters

In footer.html

```
<html><body>
<div th:fragment="copy">
  &copy; 2011 The Good Thymes Virtual Grocery
</div>
</body></html>
```

```
<div th:insert="~{footer::copy}">&copy; 2011 The Static Templates</div>
```

*{...} → Selected variable expressions

- You first select a variable with `th:object="${...}"`
- You can now reference it using `*{...}`
- Example:

```
<div th:object="${order.customer}">
  <p><b>Name:</b> <span th:text="*{name}">Frederic Tomato</span></p>
  <p><b>Since:</b> <span th:text="*{#temporals.format(customerSince, 'dd MMM
yyy')}">13 jan 2011</span></p>
</div>
```

And there's more

- Variables (`th:with`)
- Arithmetic operations
- Comparators
- Conditional HTML (`th:if`, `th:unless`)
- Iterations (`th:each`)
 - Status of iterated object (Stat suffix)
- **Call methods on objects**
 - Use with care: do not put logic in your templates!

See (referenced earlier)

- [Introduction to using thymeleaf in Spring \(start at section 5\)](#)
- <https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>



Not only for HTML...

- CSS Templates
- JavaScript Templates
- Text Templates
- XML Templates
- Raw Templates





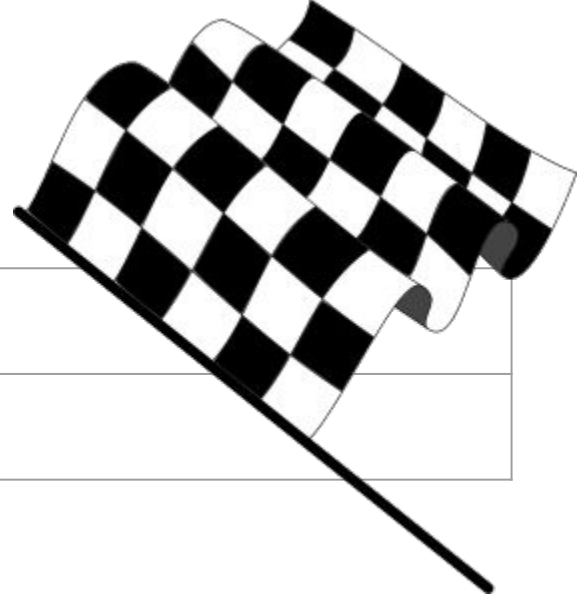
Exercise 2: Use Thymeleaf

- Download the [SMSWithThymeleaf](#) start application and inspect
- Add the following to the allstudents.html template:
 - Add a CSS file to the static folder and add a link to it in the template
 - Show the number of students in the page title: “A list of .. students”
 - Show a table of all the students
 - Add table header: Name - Birthday
 - Show name and birthday of all the students
 - Show birthday in a nice format → use Expression Objects
 - Show odd lines in the table in darker color → use Stat suffix
 - Add footer with copyright information → use a Fragment
 - Add a welcome to the user to the top left of the page → use a Fragment with a parameter
 - Add an image to the page → use @{..} to the image in the static folder
 - Provide a Resource Bundle: the page should be available in 2 languages
 - Messages: the page title, the welcome message, the table header and the footer

Agenda this week

Last week - project Review

Thymeleaf



Project

- Use Thymeleaf to work out some extra functionality to the views of your web application:
 - Use Thymeleaf fragments to show the same navigation element on all your pages
 - Add a custom CSS file in the static folder of your resources and use it (using `@{...}`) in your pages
 - *(do not spend too much time on CSS styling, next week we will learn how Bootstrap can help you with that!)*
 - Add support for a second language to your website: all static text comes from 2 different message.properties files...
 - Try experimenting with some other thymeleaf functionality:
 - Use `th:class` to highlight some rows in your table depending on the value of certain fields (eg. rows with books with more than 500 pages are red)
 - Use thymeleafs `#lists` utility object to show the number of items in the table underneath it

If you have finished the week 3 and week 4 assignments, commit it to your gitlab repo and tag it as **sprint2**

